Grammar-based Metamorphic Property Generation for NLP Models

Integrating CheckList and Fuzzing for Automated Property Generation

Jiaqi Ge

February 2026

Abstract

Test-set accuracy is the dominant metric for evaluating NLP models, but it offers limited insight into how models fail under input variation. Inspired by CheckList's metamorphic testing approach, this report extends CheckList's standard workflow in two ways. First, we introduce multi-step transformation chains ($s_0 \to s_1 \to \cdots \to s_L$) to capture cumulative and interaction effects of successive perturbations. Second, we automate test generation using grammar-based fuzzing, enabling scalable exploration beyond manual test design. Applied to a sentiment analysis task, our approach reveals failure patterns that single-step perturbations miss. We compare with original CheckList and discuss limitations including semantic drift and property validity under long chains. Code is available at https://github.com/Gracegejiaqi/metamorphic-testing-nlp.

Contents

1 Introduction

Test-set accuracy has long been the dominant metric for evaluating machine learning models, but it primarily reflects performance on standard inputs and provides limited insight into error patterns that may arise under input variation. A model can achieve high accuracy on a held-out test set yet fail systematically when confronted with small changes in wording, negation, tone, or syntactic restructuring. These failures are often invisible to aggregate accuracy measures, making them difficult to anticipate and debug. In software engineering, behavioural testing techniques — and metamorphic testing in particular — address a similar gap by checking whether a program's output changes as expected when its input is transformed in a meaningful way. CheckList brought this idea to natural language processing (NLP), offering a framework and tooling for testing models around the definition of language-related metamorphic properties. In CheckList, the developer expresses expectations on correct behaviour as an expected relationship between outputs (e.g., classifications) corresponding to families of related inputs. The framework supports the construction of perturbations (input modifications) as well as different types of output relationships, including invariance and directionality.

In this report, we extend the CheckList-style metamorphic testing workflow in two complementary directions. First, instead of applying a single perturbation to each input, we allow for the construction of multi-step transformation chains $s_0 \to s_1 \to \cdots \to s_L$, enabling the exploration of cumulative and interaction effects between successive perturbations. This long-chain setting provides broader coverage of transformed inputs and exposes failure patterns that are unlikely to surface under single-step perturbations, where only one transformation is applied in isolation. By chaining perturbations, we can simulate more realistic and complex input variations that a deployed model might encounter, such as multiple edits in a user-generated sentence or cascading effects from preprocessing errors.

Second, to make this extended workflow practical and scalable, we automate both transformation-chain generation and metamorphic property selection using grammar-based fuzzing from *The Fuzzing Book*. Manual test design, as in the original CheckList, is valuable but does not scale well to long chains where the space of possible transformation sequences grows combinatorially. Grammar-based fuzzing allows us to systematically generate diverse and syntactically valid transformation chains, as well as to select appropriate metamorphic properties for each chain, reducing human effort while increasing test coverage.

We instantiate the proposed approach on a sentiment analysis task using the Twitter US Airline Sentiment dataset and a HuggingFace sentiment classification model. Our evaluation combines quantitative summaries with qualitative failure case studies to illustrate the types of errors revealed by long transformation chains. Finally, we compare our approach with the original CheckList methodology and discuss limitations, including semantic drift (where repeated transformations gradually alter the intended meaning) and the validity of metamorphic properties under extended transformation sequences, where the expected relationship between outputs may no longer hold

even for a perfectly accurate model.

A central goal of training NLP models is to generalize beyond the training distribution. Standard evaluation practice relies on train/validation/test splits and reports aggregate statistics such as accuracy or F1. However, aggregate metrics can create an illusion of robustness: a model may achieve 90% accuracy on a test set yet fail systematically on specific subgroups, syntactic constructions, or rare but meaningful inputs. Because accuracy averages over all examples, it hides performance disparities—such as poor handling of negation, named entity variations, or linguistic perturbations—that only emerge when we examine model behavior under controlled conditions. Ribeiro et al. propose CheckList as a task-agnostic behavioral testing methodology to address this gap, shifting focus from global scores to targeted capability tests. This report adopts the same philosophy: rather than asking whether a model obtains a high score on a benchmark, we ask whether its *behavior* is consistent with expected linguistic capabilities under controlled input transformations.

1.1 Behavioral testing and metamorphic testing

Behavioral testing (also known as black-box testing) evaluates a model solely through its input–output behavior, without accessing internal parameters or architectures. This approach is particularly useful in machine learning, where models are often treated as opaque functions. However, a fundamental challenge in behavioral testing is the *oracle problem*: given a specific input, what is the correct output? In many real-world NLP tasks, the expected output is not uniquely defined or easily enumerable.

Metamorphic testing addresses the oracle problem by shifting the focus from absolute correctness to *relational correctness*. Instead of checking a single output against a ground truth, metamorphic testing defines *metamorphic relations*—constraints that should hold across multiple executions of the model. For example, if we apply a meaning-preserving transformation to an input (e.g., punctuation normalization, passive-to-active voice conversion while preserving semantics), the model's prediction should remain unchanged. Conversely, if we apply a meaning-altering transformation with a known direction (e.g., adding explicit negation to a positive sentence), the model's output should change in a predictable way (e.g., from positive to negative).

In NLP, such transformations map naturally to metamorphic relations. Meaning-preserving transformations test for robustness and invariance, while meaning-altering transformations test for sensitivity to specific linguistic phenomena. Together, they enable systematic behavioral evaluation without requiring exhaustive labeled data or access to model internals.

1.2 Main idea and contributions

Our work addresses a practical limitation in current behavioral testing practices. In standard CheckList-style workflows, most perturbation tests are applied in a *single-step* manner: each test case modifies the input exactly once (e.g., change a name, add a typo, or flip a negation) and checks the model's response. However, real-world input variation is rarely isolated. User-generated text accumulates multiple, interacting changes over time—typos, entity substitutions, formatting shifts, inserted phrases—and model failures may only surface when these perturbations compound. A model that handles each transformation individually may still fail catastrophically when they appear together.

To address this gap, we make the following contributions:

- A **multi-step transformation chain** framework. Each input is expanded into a sequence $s_0, s_1, \ldots, s_L$, where s_{i+1} is derived from s_i via a meaning-preserving or meaning-altering operation. The framework supports property checking at arbitrary depths, with particular emphasis on end-to-end comparisons between s_0 (original) and s_L (final transformed version).
- A **grammar-based fuzzing** approach for automatic test generation. Rather than manually designing perturbation pipelines, our method synthesizes long transformation chains and candidate metamorphic properties systematically, reducing human bias and expanding test coverage.
- An end-to-end **integration** with CheckList's `INV` and `DIR` test abstractions, wrapped around a HuggingFace model interface. We validate the framework through a sentiment analysis case study, including qualitative error analysis of failure patterns uncovered by multi-step perturbations.

2 Background

2.1 CheckList: capability-by-test-type structure

CheckList proposes a structured way to organize behavioral tests along two orthogonal dimensions: *linguistic capabilities* and *test types*. As described by Ribeiro et al., tests can be conceptually viewed as populating a matrix, where rows correspond to different linguistic capabilities (e.g., vocabulary, robustness, named entity recognition, negation) and columns correspond to different types of tests. This organization encourages systematic coverage of model behaviors across capabilities and test categories.

The main test types considered in CheckList include:

- **MFT** (Minimum Functionality Tests): simple, labeled examples designed to verify whether a model exhibits a basic expected behavior.
- **INV** (Invariance tests): tests in which a label-preserving perturbation is expected not to affect

the model's prediction.

- **DIR** (Directional Expectation tests): tests in which a perturbation is expected to change the prediction in a known and specified direction.

To instantiate these test types, CheckList relies on reusable perturbation operators that generate transformed inputs from existing text. Common perturbation categories include:

- **Surface-level perturbations**: transformations that modify the surface form of the input without changing its core meaning, such as punctuation variation, spelling errors, or contraction changes.
- **Lexical substitutions**: replacing words or entities with alternatives from the same category (e.g., person names, locations, numbers), typically intended to preserve or control semantic content.
- **Semantic operators**: transformations with a known semantic effect, such as adding or removing negation, which are commonly used to construct directional expectation tests.

Rather than prescribing concrete test instances, CheckList emphasizes abstractions that facilitate scalable test creation, including templates, lexicons, and reusable perturbation functions, enabling systematic behavioral testing without reliance on large manually annotated datasets.

2.2 Metamorphic testing and the oracle problem

In many settings, no reliable oracle exists to determine the correct output for arbitrary inputs. Metamorphic testing avoids this by checking relations: for a transformation T, we test whether the model output respects a specified relation between $f(x)$ and $f(T(x))$ (e.g., invariance, monotonicity, label change).

2.3 Grammar-based fuzzing

Grammar-based fuzzing generates valid test inputs by randomly sampling from a set of user-defined grammatical rules. Instead of manually enumerating test cases, a grammar specifies which elements can appear and how they can be combined, allowing the automatic generation of diverse yet well-formed inputs. The Fuzzing Book provides a practical implementation through `GrammarFuzzer`, which supports controlled random generation under simple constraints such as minimum and maximum generation depth.

In this work, we use grammar-based fuzzing to automate the construction of both *perturbation pipelines* and *metamorphic properties*. By treating sequences of perturbation operations and property specifications as grammar-generated languages, we enable scalable and systematic exploration of multi-step test scenarios beyond manual design.

3 Method: Grammar-based Multi-step Metamorphic Testing

3.1 Perturbation operator set

We reuse CheckList perturbation primitives, including:

- `add_typos`, `punctuation`, `strip_punctuation`
- `contractions`
- `change_names`, `change_location`, `change_number`
- `add_negation`, `remove_negation`

Some operators are string-based; others use spaCy parsing to locate entities or syntactic contexts.

3.2 Grammar for perturbation pipelines

We define a grammar whose language is the set of token sequences describing pipelines:

$$\langle steps \rangle \to \langle step \rangle \mid \langle step \rangle \ \langle steps \rangle.$$

Sampling from this grammar yields a variable-length pipeline π with L steps.

3.3 Grammar for property generation

We define a second grammar for the property space. In the simplest setting, we constrain $k = L$ and generate exactly one property per run:

$$\langle prop \rangle \to \text{INV\_}FINAL(k) \mid \text{DIR\_}FINAL(k, \texttt{EXPECT} = \cdot).$$

3.4 CheckList integration

Given chains $\{(s_0^{(i)}, s_k^{(i)})\}_{i=1}^n$, we instantiate CheckList tests:

- INV test with data pairs $(s_0^{(i)}, s_k^{(i)})$.
- DIR test with a CheckList expectation function (pairwise comparison between original and transformed probabilities/labels).

3.5 Algorithm summary

Algorithm 1 Grammar-based Multi-step Metamorphic Testing

Require: Dataset texts $\{s_0^{(i)}\}_{i=1}^{n}$, perturbation grammar G_{steps}, property grammar G_{prop}

Sample pipeline $\pi = (T_1, \ldots, T_L) \sim G_{\text{steps}}$
for $i = 1$ to n **do**
    $s \leftarrow s_0^{(i)}$
    for $j = 1$ to L **do**
        $s \leftarrow T_j(s)$
        store as $s_j^{(i)}$
    end for
end for
Sample property $P \sim G_{\text{prop}}$ (typically with $k = L$)
Instantiate CheckList test suite from pairs $(s_0^{(i)}, s_k^{(i)})$ and run on model
Perform qualitative analysis by extracting representative passing/failing pairs

4 Implementation Details

4.1 Data loading

We use the Twitter US Airline Sentiment dataset and take the first 100 tweet texts as seed inputs. Each input is converted to a string and placed at chain index 0.

4.2 Perturbation application

Each operator token is mapped to a concrete CheckList perturbation function. Some perturbations return multiple candidates; for simplicity we choose the first available transformed output, falling back to the original sentence if transformation fails.

4.3 Model wrapper

We wrap a HuggingFace sentiment model using CheckList's `PredictorWrapper.wrap_softmax`. Outputs are converted into a two-class probability vector consistent with our labels.

4.4 Directional expectations

We implement multiple DIR expectations:

- `TO_NEG_IF_POS` and `TO_POS_IF_NEG`: conditional label flips.

- `MORE_NEGATIVE` / `MORE_POSITIVE`: confidence monotonicity on a specific class.
- `CHANGE_LABEL`: any label change.

5 Experimental Setup

5.1 Dataset

Twitter US Airline Sentiment is a labeled dataset of tweets collected around February 2015 about major U.S. airlines. It includes sentiment labels and metadata; we use the `text` field as unlabeled seed inputs for perturbation-based testing. We also rely on the dataset in CSV form as hosted on the Hugging Face datasets hub.

5.2 Model

We use a HuggingFace `pipeline("sentiment-analysis")` model. When no model is explicitly specified, the pipeline defaults to a DistilBERT checkpoint fine-tuned on SST-2. This choice matches the behavior observed in the execution log and provides an out-of-the-box baseline.

5.3 Metrics and reporting

We report:

- **Transformation coverage per step:** number of samples changed at step j.
- **Property outcomes:** proportion of pairs satisfying the generated INV/DIR property (when available from CheckList test APIs).
- **Qualitative analysis:** representative passing/failing examples with probability shifts.

6 Results

6.1 Example run: a 9-step pipeline

In one representative run, the sampled pipeline contains $L = 9$ steps:

(`add_negation`, `punctuation`, `punctuation`, `remove_negation`, `add_negation`, `add_negation`, `change_locati`

The number of changed samples per step over $n = 100$ seeds is summarized in Table 1.

Table 1: Per-step change counts for a sampled 9-step transformation chain ($n = 100$).

Step	Operator	Changed samples
1	add_negation	74/100
2	punctuation	100/100
3	punctuation	100/100
4	remove_negation	81/100
5	add_negation	84/100
6	add_negation	27/100
7	change_location	4/100
8	add_typos	96/100
9	add_negation	18/100

6.2 Generated property and outcomes

For the same run, the property grammar sampled:

$$\text{DIR\_}FINAL(9, \texttt{EXPECT} = \texttt{MORE\_NEGATIVE}),$$

meaning the model's negative probability should increase from s_0 to s_9.

Interestingly, qualitative examples show both:

- **Expected direction holds** when adding negation flips or reduces positivity.
- **Counterintuitive failures** where additional negation or formatting changes produce label flips to positive or reduce negative confidence, suggesting non-linear interactions.

6.3 Qualitative case study

Below we highlight representative pairs (original s_0 vs. final s_9) illustrating failure modes:

Failure example (direction violated): the transformed sentence introduces a negation pattern that changes discourse structure; the model becomes *less* negative (or even positive), contradicting `MORE_NEGATIVE`.

Passing example (direction holds): explicit negation like "I don't love ..." yields a strong increase in negative confidence, matching expectation.

(Complete logs and additional examples are provided in the Appendix.)

7 Comparison with the Original CheckList Workflow

7.1 What CheckList already provides

CheckList offers a structured methodology (capabilities × test types) and a toolchain for generating many test cases quickly, using templates, lexicons, and perturbation functions. In practice, many tests are executed as single-step perturbations (e.g., swap a location name; add a negative phrase) and check invariance or directional expectation on the resulting pair.

7.2 Our extension: long chains and fuzzed properties

Our approach differs in two key aspects:

- **From pair to chain:** instead of comparing only (s_0, s_1), we can compare (s_0, s_k) for arbitrary k and systematically study accumulation and interaction.
- **From manual to grammar-fuzzed exploration:** instead of manually enumerating perturbations and expectations, we sample both pipelines and properties from grammars, enabling broader search over test scenarios.

This is not a replacement for CheckList. Rather, it is a complementary "test generation" layer: once a chain and property are sampled, CheckList remains the execution and reporting engine.

7.3 Why multi-step matters

Single-step tests are excellent for diagnosing isolated capabilities (e.g., whether name changes should preserve sentiment). However, real inputs often undergo multiple changes: typos, formatting, entity variations, and semantic modifiers may co-occur. A long chain can reveal:

- **Interaction bugs:** two perturbations individually benign may jointly cause unexpected failure.
- **Cumulative drift:** invariance may degrade gradually rather than abruptly.
- **Robustness boundaries:** the "distance" (number of perturb steps) a model can tolerate before behavior changes.

8 Threats to Validity and Limitations

8.1 Semantic drift

Not all multi-step chains preserve the original ground-truth sentiment. As L grows, label validity can change, making INV properties less meaningful. Directional properties can also fail when

transformations introduce contradictory semantics (e.g., double negation patterns, scope ambiguity).

8.2 Grammar bias

The fuzzing grammar defines the search space. If the grammar over-represents punctuation or negation, tests will be skewed toward those phenomena. A principled grammar design (or adaptive grammar learning) is future work.

9 Conclusion

In this report, we addressed a practical limitation of standard behavioral testing for NLP models: the predominant reliance on single-step perturbations, which fails to capture how multiple, interacting input changes accumulate in real-world scenarios. To overcome this, we introduced a multi-step transformation chain framework that expands each seed input into a sequence $s_0, s_1, \ldots, s_L$, enabling systematic evaluation of cumulative and interaction effects across successive perturbations.

To make this extension practical and scalable, we further contributed a grammar-based fuzzing approach that automatically synthesizes perturbation pipelines and candidate metamorphic properties, reducing manual test design effort and expanding test coverage beyond what is feasible by hand. We integrated these contributions with CheckList's existing `INV` and `DIR` abstractions, preserving compatibility with the original workflow while adding a new generative layer.

Our sentiment analysis case study demonstrated that multi-step chains can uncover failure patterns invisible to single-step tests. In particular, we observed cases where individually benign perturbations—such as punctuation normalization and name substitution—produced unexpected label changes or violated directional expectations when combined, highlighting the importance of testing interaction effects. The qualitative error analysis further revealed that even meaning-preserving transformations can lead to semantic drift over long chains, a phenomenon that existing metamorphic testing literature has largely overlooked.

Our comparison with the original CheckList workflow clarified that our approach is not a replacement but a complementary test generation layer: once a chain and property are sampled, CheckList remains the execution and reporting engine. This division of labor preserves the usability and interpretability of CheckList while extending its reach.

Limitations and Future Work

Despite these contributions, our method has several limitations that point to promising future directions. First, **semantic drift** remains a fundamental challenge: as chain length grows, the ground-truth label of the transformed input may diverge from that of the original, rendering both

invariance and directional expectations invalid even for a perfectly accurate model. Future work could incorporate semantic similarity constraints (e.g., using embedding-based distance thresholds) or chain-specific property annotations to filter or correct for drift.

Second, our **grammar design introduces bias**: the current grammar over-represents certain perturbation types (e.g., negation, punctuation) while under-representing others (e.g., syntactic voice changes, dialectal shifts). A more principled approach might learn grammars from corpus data or adaptively expand the grammar based on observed failure modes.

Third, we currently sample perturbation pipelines and properties *independently*, but the validity of a property depends on the semantics of the applied transformation chain. For example, `MORE_NEGATIVE` is meaningful after adding negation but nonsensical after removing it. Future work could enforce *semantic consistency constraints* between pipelines and properties, either through grammar co-design or post-hoc validation heuristics.

Finally, our evaluation was limited to a single sentiment analysis model and dataset. Broader validation across tasks (e.g., natural language inference, toxicity detection) and model architectures (e.g., large language models with different training distributions) would strengthen the generality of our findings.

In summary, this report provides a first step toward automated, multi-step metamorphic testing for NLP models. By exposing interaction failures and cumulative robustness boundaries, grammar-based chain generation opens new opportunities for systematic behavioral evaluation—an increasingly critical complement to accuracy-focused benchmarking as NLP models are deployed in complex, dynamic environments.

A Appendix: Full Source Code (Python)

```
import re
import spacy
import pandas as pd
import torch
from fuzzingbook.GrammarFuzzer import GrammarFuzzer
from transformers import pipeline

from checklist.perturb import Perturb
from checklist.test_types import INV, DIR
from checklist.pred_wrapper import PredictorWrapper
from checklist.expect import Expect


df = pd.read_csv("hf://datasets/osanseviero/twitter-airline-sentiment/Tweets.csv")
sentences = df["text"].astype(str).tolist()[:100]
print("Loaded examples:", sentences[:5])

nlp = spacy.load("en_core_web_sm")


PERTURB_STEPS_GRAMMAR = {
    "<start>": ["<steps>"],
    "<steps>": ["<step>", "<step> <steps>"],
    "<step>": [
        "add_typos",
        "punctuation",
        "strip_punctuation",
        "contractions",
        "change_names",
        "change_location",
        "change_number",
        "add_negation",
        "remove_negation",
    ],
}

seq_fuzzer = GrammarFuzzer(PERTURB_STEPS_GRAMMAR, min_nonterminals=3, max_nonterminals
    =50)
ops = seq_fuzzer.fuzz().split()
print("Random pipeline steps:", ops)
```

```
def apply_one_op(s: str, token: str) -> str:
    if token == "add_typos":
        r = Perturb.perturb([s], Perturb.add_typos)

    elif token == "contractions":
        r = Perturb.perturb([s], Perturb.contractions)

    elif token == "punctuation":
        r = Perturb.perturb([nlp(s)], Perturb.punctuation)

    elif token == "strip_punctuation":
        r = Perturb.perturb([nlp(s)], Perturb.strip_punctuation)

    elif token == "change_names":
        r = Perturb.perturb([nlp(s)], Perturb.change_names, n=3)

    elif token == "change_location":
        r = Perturb.perturb([nlp(s)], Perturb.change_location, n=3)

    elif token == "change_number":
        r = Perturb.perturb([nlp(s)], Perturb.change_number, n=3)

    elif token == "add_negation":
        r = Perturb.perturb([nlp(s)], Perturb.add_negation)

    elif token == "remove_negation":
        try:
            new_s = Perturb.remove_negation(nlp(s))
            return new_s if new_s and new_s != s else s
        except Exception:
            return s

    else:
        return s

    if r is None or not hasattr(r, "data") or not r.data:
        return s

    new_list = r.data[0][1:]
    if not new_list:
        return s

    out = new_list[0]
    return out if out else s
```

```
chains = [[s] for s in sentences]

for step_idx, op in enumerate(ops, start=1):
    print(f"\nApplying step S{step_idx}: {op}")
    num_changed = 0

    for i in range(len(chains)):
        current = chains[i][-1]
        new = apply_one_op(current, op)
        if new != current:
            num_changed += 1
        chains[i].append(new)

    print(f"S{step_idx}: changed {num_changed} / {len(chains)} samples")

L = len(ops)
print(f"\nTotal steps in this pipeline: {L}")


K_CHOICES = [str(L)]
PROPERTY_GRAMMAR = {
    "<start>": ["<prop>"],
    "<prop>": [
        "INV_FINAL(<k>)",
        "DIR_FINAL(<k>,EXPECT=TO_NEG_IF_POS)",
        "DIR_FINAL(<k>,EXPECT=TO_POS_IF_NEG)",
        "DIR_FINAL(<k>,EXPECT=MORE_NEGATIVE)",
        "DIR_FINAL(<k>,EXPECT=MORE_POSITIVE)",
        "DIR_FINAL(<k>,EXPECT=CHANGE_LABEL)",
    ],
    "<k>": K_CHOICES,
}

prop_fuzzer = GrammarFuzzer(PROPERTY_GRAMMAR, min_nonterminals=1, max_nonterminals=5)
prop = prop_fuzzer.fuzz().strip()
props = [prop]
print("\nGrammar-picked property:", props)


device = 0 if torch.backends.mps.is_available() else -1
clf = pipeline("sentiment-analysis", device=device)
```

```
def hf_softmax(xs):
    texts = [x[-1] if isinstance(x, (tuple, list)) else x for x in xs]
    outputs = clf(texts, truncation=True)

    probs = []
    for o in outputs:
        label = o["label"].upper()
        score = float(o["score"])
        if label.startswith("NEG"):
            probs.append([score, 1.0 - score])
        else:
            probs.append([1.0 - score, score])
    return probs


wrapped = PredictorWrapper.wrap_softmax(hf_softmax)


tests = []

re_inv = re.compile(r"^INV_FINAL\((\d+)\)$")
re_dir = re.compile(r"^DIR_FINAL\((\d+),EXPECT=([A-Z_]+)\)$")


def build_dir_expect(expect_name: str):
    eps = 1e-12

    if expect_name == "TO_NEG_IF_POS":
        e = Expect.eq(0)
        return Expect.slice_orig(e, lambda orig, *args: orig == 1)

    if expect_name == "TO_POS_IF_NEG":
        e = Expect.eq(1)
        return Expect.slice_orig(e, lambda orig, *args: orig == 0)

    if expect_name == "MORE_NEGATIVE":
        def more_neg(orig_pred, pred, orig_conf, conf, labels=None, meta=None):
            try:
                return float(conf[0]) > float(orig_conf[0]) + eps
            except Exception:
                return None
        return Expect.pairwise(more_neg)
```

```
    if expect_name == "MORE_POSITIVE":
        def more_pos(orig_pred, pred, orig_conf, conf, labels=None, meta=None):
            try:
                return float(conf[1]) > float(orig_conf[1]) + eps
            except Exception:
                return None
        return Expect.pairwise(more_pos)

    if expect_name == "CHANGE_LABEL":
        def changed_pred(orig_pred, pred, orig_conf, conf, labels=None, meta=None):
            return pred != orig_pred
        return Expect.pairwise(changed_pred)

    raise ValueError(f"Unknown DIR expectation: {expect_name}")


for p in props:
    m = re_inv.match(p)
    if m:
        k = int(m.group(1))
        data_k = [[c[0], c[k]] for c in chains]
        tests.append(INV(
            data=data_k,
            name=f"INV final-only k={k} (grammar)",
            capability="Robustness",
        ))
        continue

    m = re_dir.match(p)
    if m:
        k = int(m.group(1))
        expect_name = m.group(2)
        data_k = [[c[0], c[k]] for c in chains]
        tests.append(DIR(
            data=data_k,
            expect=build_dir_expect(expect_name),
            name=f"DIR final-only k={k}, expect={expect_name} (grammar)",
            capability="Robustness",
            description="Metamorphic property generated by grammar; results are for user
                 interpretation.",
        ))
        continue

print(f"\nBuilt {len(tests)} tests: grammar-picked INV/DIR only.")
```

```
re_dir_name = re.compile(r"^DIR final-only k=(\d+), expect=([A-Z_]+)")
re_inv_name = re.compile(r"^INV final-only k=(\d+)")

def _pred_label(prob):
    return 0 if prob[0] >= prob[1] else 1

def _lab(x):
    return "NEG" if x == 0 else "POS"

def _fmt(prob):
    return f"{prob[0]:.4f},{prob[1]:.4f}({_lab(_pred_label(prob))})"

def _dir_considered_and_pass(expect_name, orig_prob, prob, eps=1e-12):
    orig_pred = _pred_label(orig_prob)
    pred = _pred_label(prob)

    if expect_name == "TO_NEG_IF_POS":
        considered = (orig_pred == 1)
        passed = (pred == 0)
        return considered, passed

    if expect_name == "TO_POS_IF_NEG":
        considered = (orig_pred == 0)
        passed = (pred == 1)
        return considered, passed

    if expect_name == "MORE_NEGATIVE":
        try:
            return True, (float(prob[0]) > float(orig_prob[0]) + eps)
        except Exception:
            return True, False

    if expect_name == "MORE_POSITIVE":
        try:
            return True, (float(prob[1]) > float(orig_prob[1]) + eps)
        except Exception:
            return True, False

    if expect_name == "CHANGE_LABEL":
        return True, (pred != orig_pred)

    return False, False
```

```
def print_dir_examples(test_name, n_fail=10, n_pass=10, batch_size=64):
    m = re_dir_name.match(test_name)
    if not m:
        return
    k = int(m.group(1))
    expect_name = m.group(2)

    fails = []
    passes = []
    filtered = 0

    n = len(chains)
    start = 0
    while start < n and (len(fails) < n_fail or len(passes) < n_pass):
        end = min(start + batch_size, n)

        orig_texts = [chains[i][0] for i in range(start, end)]
        pert_texts = [chains[i][k] for i in range(start, end)]

        orig_probs = hf_softmax(orig_texts)
        pert_probs = hf_softmax(pert_texts)

        for j in range(len(orig_texts)):
            op = orig_probs[j]
            pp = pert_probs[j]
            considered, passed = _dir_considered_and_pass(expect_name, op, pp)
            if not considered:
                filtered += 1
                continue

            rec = (op, pp, orig_texts[j], pert_texts[j])

            if passed and len(passes) < n_pass:
                passes.append(rec)
            if (not passed) and len(fails) < n_fail:
                fails.append(rec)

            if len(fails) >= n_fail and len(passes) >= n_pass:
                break

        start = end

    print(f"\nFiltered out (orig not applicable): {filtered}")
```

```
    print(f"\nExample fails (showing {len(fails)}/{n_fail}):")
    for op, pp, s0, sk in fails:
        print(f"{_fmt(op)} {s0}")
        print(f"{_fmt(pp)} {sk}")
        print("\n----")

    print(f"\nExample passes (showing {len(passes)}/{n_pass}):")
    for op, pp, s0, sk in passes:
        print(f"{_fmt(op)} {s0}")
        print(f"{_fmt(pp)} {sk}")
        print("\n----")

def print_inv_examples(test_name, n_fail=10, n_pass=10, batch_size=64):
    m = re_inv_name.match(test_name)
    if not m:
        return
    k = int(m.group(1))

    fails = []
    passes = []

    n = len(chains)
    start = 0
    while start < n and (len(fails) < n_fail or len(passes) < n_pass):
        end = min(start + batch_size, n)

        orig_texts = [chains[i][0] for i in range(start, end)]
        pert_texts = [chains[i][k] for i in range(start, end)]

        orig_probs = hf_softmax(orig_texts)
        pert_probs = hf_softmax(pert_texts)

        for j in range(len(orig_texts)):
            op = orig_probs[j]
            pp = pert_probs[j]
            orig_pred = _pred_label(op)
            pred = _pred_label(pp)

            rec = (op, pp, orig_texts[j], pert_texts[j])

            if (pred == orig_pred) and len(passes) < n_pass:
                passes.append(rec)
            if (pred != orig_pred) and len(fails) < n_fail:
                fails.append(rec)
```

```
            if len(fails) >= n_fail and len(passes) >= n_pass:
                break

        start = end

    print(f"\nExample fails (showing {len(fails)}/{n_fail}):")
    for op, pp, s0, sk in fails:
        print(f"{_fmt(op)} {s0}")
        print(f"{_fmt(pp)} {sk}")
        print("\n----")

    print(f"\nExample passes (showing {len(passes)}/{n_pass}):")
    for op, pp, s0, sk in passes:
        print(f"{_fmt(op)} {s0}")
        print(f"{_fmt(pp)} {sk}")
        print("\n----")


for test in tests:
    print(f"\n===== {test.name} =====")
    test.run(wrapped)

    if test.name.startswith("DIR final-only"):
        print_dir_examples(test.name, n_fail=10, n_pass=10)

    elif test.name.startswith("INV final-only"):
        print_inv_examples(test.name, n_fail=10, n_pass=10)
```

B Appendix: Example Execution Log

```
Loaded examples: ['@VirginAmerica What @dhepburn said.', ...]
Random pipeline steps: ['add_negation', 'punctuation', 'punctuation', 'remove_negation'
    ,
                        'add_negation', 'add_negation', 'change_location', 'add_typos', '
                            add_negation']

Applying step S1: add_negation
S1: changed 74 / 100 samples
...
Applying step S9: add_negation
S9: changed 18 / 100 samples

Total steps in this pipeline: 9
Grammar-picked property: ['DIR_FINAL(9,EXPECT=MORE_NEGATIVE)']
Built 1 tests: grammar-picked INV/DIR only.
===== DIR final-only k=9, expect=MORE_NEGATIVE (grammar) =====
Predicting 200 examples
...
```

References

[1] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. https://arxiv.org/abs/2005.04118

[2] Marco Tulio Ribeiro. CheckList GitHub Repository. https://github.com/marcotcr/checklist

[3] Andreas Zeller, Rahul Gopinath, Marcel Böhme, Gordon Fraser, and Christian Holler. The Fuzzing Book: Tools and Techniques for Generating Software Tests. https://www.fuzzingbook.org/.

[4] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic Testing: A New Approach for Generating Next Test Cases. arXiv:2002.12543, 2020. https://arxiv.org/abs/2002.12543.